

Memoria Proyecto Final

Memoria Proyecto Final.....	1
1. Resumen del trabajo.....	2
2. Objetivos.....	2
2.1 Funcionalidades implementadas.....	2
3. Diseño/Implementación.....	3
3.1 Simplificaciones.....	3
3.2 Guía de Usuario.....	3
Cómo instalar el plugin.....	3
Cómo extraer las cadenas de texto.....	4
Cómo iniciar el sistema en ejecución.....	5
Cómo usar UI Clamping.....	7
Cómo utilizar los modificadores de género.....	9
Cómo utilizar el sistema de variables.....	10
Cómo utilizar el sistema monetario.....	11
Cómo utilizar sistema de cantidades (singulares y plurales).....	12
3.3 Implementación.....	14
Diagrama de Clases.....	14
Archivos XML.....	14
4. Resultados Obtenidos.....	16
4.1 Realización de pruebas.....	16
4.2 Cambios en la extraordinaria.....	16
5. Conclusiones.....	17
6. Adenda.....	18
6.1 Aportaciones en la ordinaria.....	18
6.2 Aportaciones en la extraordinaria.....	20

1. Resumen del trabajo

Unity tiene implementado su propio sistema de Internacionalización. Nosotros queremos hacer nuestro propio sistema de internacionalización mediante un plugin de Unity que se pueda usar en varios juegos.

Usaremos este [repositorio](#) y este [Drive](#).

2. Objetivos

Nuestro objetivo principal es implementar dicho sistema como plugin de Unity. Además, lo probaremos dos juegos para comprobar que funciona: “Civilízame Esta” y “El custodio de Babel” [[pruebas](#)]

2.1 Funcionalidades implementadas

- Extracción de cadenas de texto del editor (strings en objetos Serializables o componentes TMP_Text) automáticamente a un mapa con claves diferente para un idioma
- Lectura/escritura de XML que adecue el texto a los diferentes idiomas
- Variables dentro del texto (nombre del jugador, tipos de objeto, etc)
- Modificadores de género
- Archivos de configuración de idioma:
 - Dirección del texto
 - Símbolo decimal
 - Símbolo separador de mil unidades
 - Símbolo de dinero
- Compatibilidad con los modificadores de tamaño de las UIs para que los textos se mantengan dentro de un margen y no se salgan.
- Una interfaz con un botón para facilitar la extracción de las cadenas de texto.

3. Diseño/Implementación

3.1 Simplificaciones

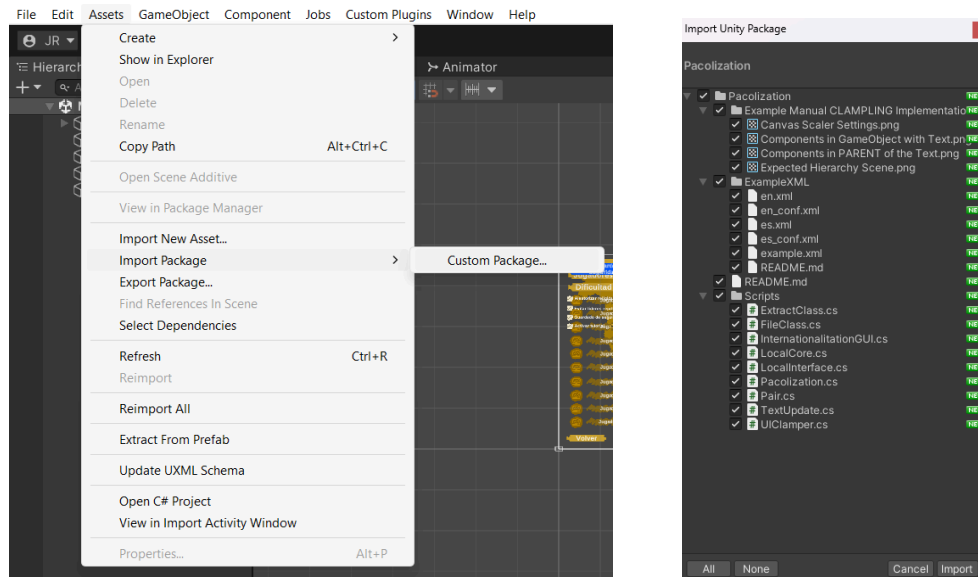
Para evitar algunas complicaciones durante el diseño y programación, se han asumido las siguientes simplificaciones del diseño que el usuario tendrá que tener en cuenta

- El usuario será el encargado de llevar la cuenta del idioma en el que está y de actualizar a la clase LocalInterface cuando cambie. Además, el usuario deberá llevar la cuenta de que int corresponde a cada idioma cuando extraiga los strings o guarde los cambios en los diferentes XML de cada idioma.
- Durante la extracción de texto, vamos a extraer solamente textos de tipo TMP_Text, ignorando la clase Text(Legacy) ya que esta última resulta ya obsoleta a día de hoy.
- Asumiremos que solo leeremos y guardaremos los strings que solo están dentro de la escena y no en los assets - salvo por los scriptable objects - para evitar repeticiones y problemas de solapamientos al guardar los datos
- Las escenas de las que extraemos strings serán las escenas que están incluidas en la build del juego.
- Para buscar ScriptableObjects el usuario deberá de aportar sus propias rutas de archivo para evitar que se cojan ScriptableObjects no propios (de paquetes o de Unity)
- Existe la opción de automatizar la inclusión de componentes necesarios para el UI Clamping de los textos de la UI (que se explica más adelante), pero el usuario deberá de cambiar a mano los tamaños de los cuadros de textos para tener las dimensiones esperadas, además de adecuar el formato de los cuadros de texto (tipo de gameObjects de la UI a tener que usar) y adecuar el formato del Canvas, que se explican más adelante en la guía de usuario [\[Más info sobre los UI clampers\]](#).
- El formato de lectura de textos será de Izquierda a Derecha o de Derecha a Izquierda, Unity no tiene soporte incluido para poder poner de Arriba a Abajo y de Abajo a Arriba.
- El plugin ha sido desarrollado en Unity 6 y no aseguramos que exista retrocompatibilidad (aunque puede que exista)

3.2 Guía de Usuario

Cómo instalar el plugin

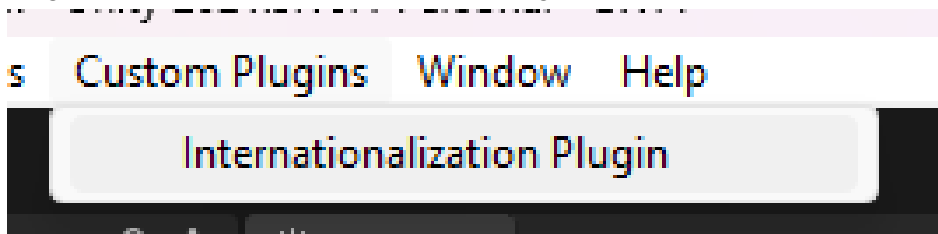
1. Primero uno debe descargar la release más reciente del repositorio de Github de Pacolization.
2. Después hay que importar el archivo "Pacolization.unpackage" dentro del proyecto de Unity en el que se quiera instalar. Esto se puede hacer haciendo click en la pestaña de Assets/Import Package/Custom Package y seleccionando el archivo, o directamente arrastrando "Pacolization.unpackage" al editor de Unity.
3. Aparecerá una pestaña "Import Unity Package" similar a la que se muestra abajo. Hay que hacerle clic al botón de abajo a la derecha en el que pone "Import"



4. Con eso el plugin ya estaría instalado. Tan solo quedaría configurarlo para realizar cada una de las tareas que se quieran hacer, tal como se indicará más adelante.

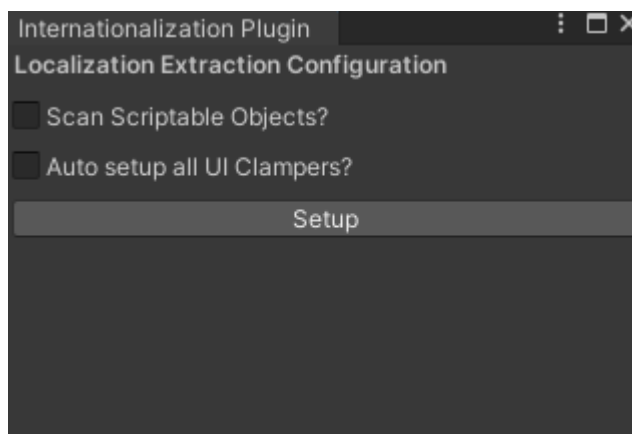
Cómo extraer las cadenas de texto

1. Abrir la pestaña de Custom Plugins en la parte superior del editor de Unity
2. Elegir la pestaña “Internationalization Plugin”



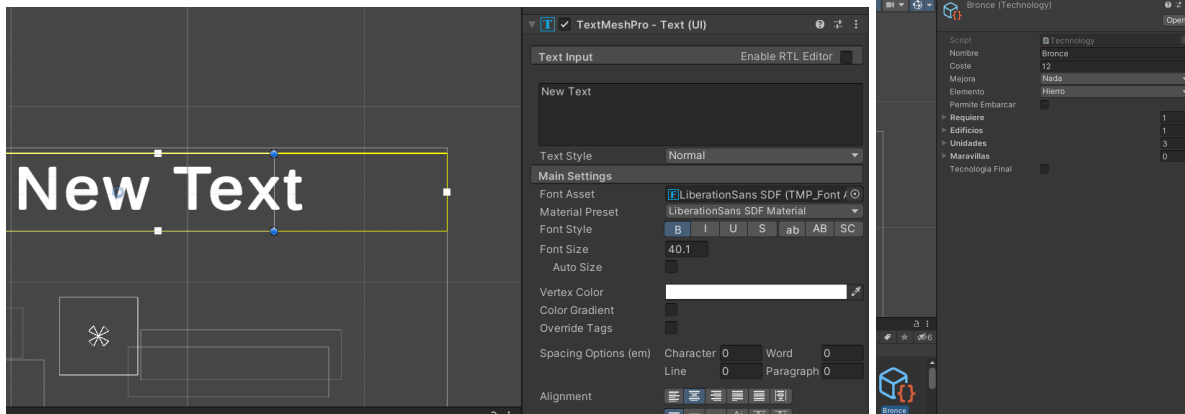
La pestaña de Custom Plugins y la selección de nuestro plugin

3. Al hacer doble clic en “Internationalization Plugin”, debería aparecer la siguiente ventana:



4. Primero, deberá elegir si la extracción afectará a Scriptable Objects. Por defecto, la extracción de cadenas solo afectará a los objetos con el componente TMP_Text que

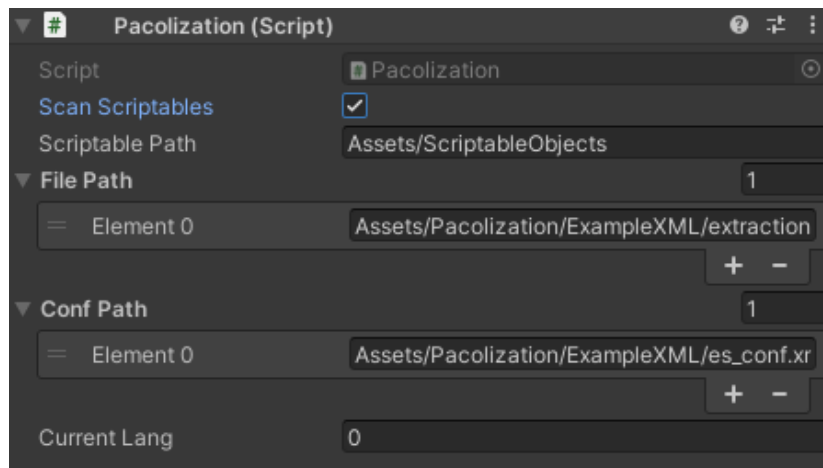
estén en la build del juego. Si se elige la opción de extraer las cadenas de Scriptable Objects, se deberá además seleccionar el directorio donde se encuentran. **CUIDADO:** No solo se escaneará el directorio seleccionado, si no también los directorios hijos. Se recomienda prudencia para evitar tocar Scriptable Objects que no necesiten ser traducidos (p. ej., archivos de guardado, datos de juego, etc.)



5. Elegir si se van a usar los UI clampers, que se aplicarán a los objetos de los cuales se extraiga el texto. [\[Más información sobre los UI clampers\]](#)
6. Pulsar el botón de extracción y esperar a que se termine el proceso.

Cómo iniciar el sistema en ejecución

1. Para poder usarse en ejecución, el usuario deberá crear un objeto vacío en la primera escena que se ejecutará cuando se lance la build.
2. A ese objeto se le deberá de añadir el componente Pacolization.
3. Después, se debe introducir en Pacolization los datos necesarios:
 - **Scan Scriptables:** para indicar si deseas escanear los scriptable objects o no.
 - **Scriptable Path:** path en el que deben encontrarse los scriptable a escanear.
 - **File Path:** array de paths desde el que se cargaran todos los archivos con los textos de cada idioma, **IMPORTANTE el índice de cada archivo con los textos de un idioma debe coincidir con el de su configuración**
 - **Conf Path:** array de paths desde el que se cargaran todos los archivos con la configuración de cada idioma, **IMPORTANTE el índice de cada archivo con los textos de un idioma debe coincidir con el de su configuración y además deben incluir los campos visibles en los ficheros de ejemplo del paquete.**
 - **CurrentLang:** índice del idioma en el que se empezarán a leer los textos y la configuración.
4. . Los campos están serializados, por lo que se podrá llevar a cabo desde el editor.



Los datos son la ruta a los ScriptableObjects (si se han usado), un array de las rutas a los archivos con los textos traducidos, y un array de las rutas a la configuración de los idiomas

Si el usuario quisiera, podría implementar por su parte dicha clase y/o añadirla a otro objeto que también vaya a seguir el patrón Singleton. De cualquier manera, se deberá tener en cuenta las mismas propiedades que implementa Pacolization (patrón singleton y **Don't Destroy on Load**) y llamar a los respectivos métodos en **Awake** y **OnQuit**.

```
LocalInterface Li;

private static Pacolization _instance;
public static Pacolization Instance()
{ return _instance; }

// Start is called once before the first execution of Update after the MonoBehaviour is created
void Awake()
{
    if (_instance == null)
    {
        _instance = this;
        DontDestroyOnLoad(this);

        Li = LocalInterface.Instance();
        Li.Initiate(scanScriptables, scriptablePath);
        Li.StartInExecution(filePath[currentLang], currentLang, confPath[currentLang]);
    }
    else
    {
        Destroy(this);
    }
}

public void changeLang(int id)
{
    currentLang =(uint) id;
    Li.changeLang(filePath[id], (uint)id, confPath[currentLang]);
}

public void WriteGenderConf(string key, int value)
{
    Li.WriteGenderConfToXML(key,value,filePath[currentLang]);
}

public void WriteVariables(string key, string value)
{
    Li.WriteVariables(key,value,filePath[currentLang]);
}

void OnApplicationQuit()
{
    LocalInterface.Instance().OnQuit();
}
```

- Una vez inicializado, el usuario se comunicará llamando a los métodos de Pacolization. Esta clase también sigue un patrón singleton y sirve como intermediario entre el usuario y el código del plugin, permitiendo acceder a toda

funcionalidad. El usuario nunca debería de comunicarse con ninguna clase del plugin aparte de Pacolization.

6. Una vez incluida la lógica para cambiar de idioma elegida por el usuario (en la cuál deberá de llevar la cuenta del idioma actual), se deberá comunicar el cambio al método `ChangeLang` de Pacolization. Este cambio es automático y podrá llevarse a cabo incluso sin necesidad de recargar la escena actual.

Cómo usar UI Clamping

Para qué sirve

A veces cuando escribimos un mismo texto en diferentes idiomas, la longitud de este puede variar y esto puede provocar que el texto se salga de los límites del cuadro, incluso que se salga de la pantalla. Para evitar esto hemos implementado una herramienta que intenta minimizar ese riesgo, el UI Clamping.

Configuración del Canvas y cómo crear bien los cuadros de textos

Los siguientes pasos son recomendados ya se vaya a tomar la ruta automática o la manual en la configuración del UI Clamping.

A la hora de crear un Canvas en una escena que contenga diferentes elementos de UI, el Canvas debe de tener estos ajustes:

- “Canvas Scaler”
 - “UI Scale Mode” — “Scale With Screen Size”
 - “Reference Resolution”: formato panorámico igual al de la configuración del juego, con Y de al menos 1080p.

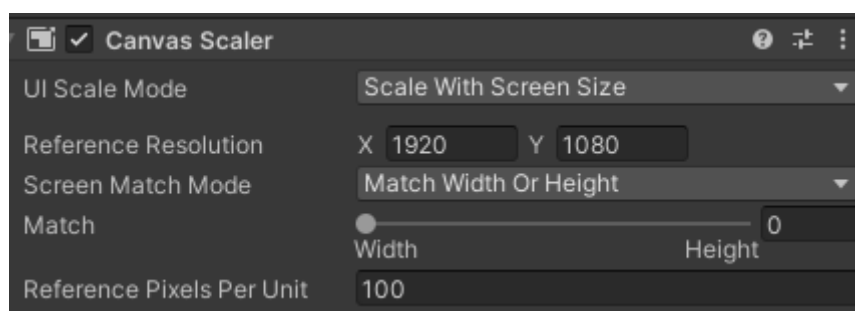


Imagen de ejemplo del componente “Canvas Scaler”

Cuidado: Si un texto suelto el cuál es objeto hijo de una imagen puede causar problemas.

Para implementar un texto con recuadro simple que no genere tantos conflictos, primero añadimos dentro del Canvas un gameobject de la UI del tipo “Image”, y en su componente de “Image” ponemos el tipo de imagen en “Sliced”.

Después al gameobject del tipo “Image” se le añade un gameobject hijo del tipo “Text Mesh Pro” para el texto. Luego se aplica el UI Clamping Automático a todos los textos y no deberían de haber tantos problemas.

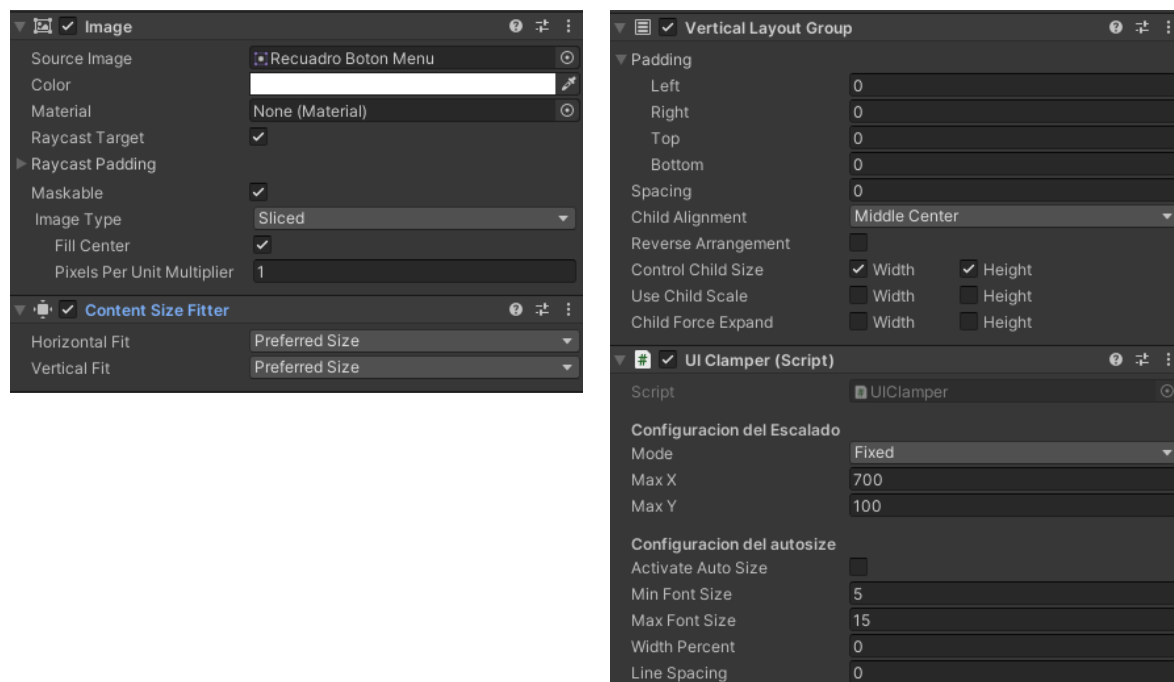
La jerarquía debería de quedar así (StartButton es el objeto del tipo “Image”):



Es necesario que todos los textos estén configurados y creados tal y como se han descrito antes. Si solo hay texto, pero no está contenido en un cuadro de texto propio como se ve en la imagen de justo arriba, no funcionará el UI Clamping, por ejemplo.

Implementación MANUAL del UI Clamping en los textos de la UI

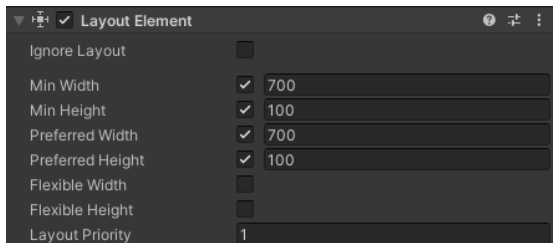
Cuando añadimos un botón o un cuadro de texto por ejemplo, este tiene un gameobject HIJO en la escena de Unity que es el texto. En el gameobject PADRE, que es donde está el cuadro de texto, debemos tener asignados los siguientes componentes (imagen sacada del inspector de Unity):



- "UIClamper": es un componente que hemos creado para este plugin, se encarga de mantener el cuadro de texto dentro de la pantalla y procura que el texto se escriba sin desbordarse del cuadro de texto. Lo que es cada variable viene comentado en el código del componente.

En el gameobject HIJO, el que contiene el propio texto, debe de añadirse este componente:

- “Layout Element”: necesario añadirlo para que el texto no se salga de la pantalla y que el componente “UI Clamper” del PADRE funcione. Si se ha puesto en el PADRE el UI Clamper, los valores de Width y Height se cambian automáticamente a los del “Max X” y “Max Y” del UI Clamper.



Implementación AUTOMÁTICA del UI Clamping en los textos de la UI

Para encontrar esta funcionalidad hay que ir a "Internationalization Plugin", donde hay una opción que pone "Auto Setup All UI Clampers". Al activar dicha opción se encargará de implementar en todos los textos de la UI el UI Clamping. Pero si el tamaño de uno de los cuadros de texto no es el que se espera tener, hay que ir al componente “UI Clamper” del gameobject PADRE y cambiar los valores de “Max X” y “Max Y”.

Cómo utilizar los modificadores de género

Los modificadores de género están implementados dentro de la lectura de los archivos de idiomas. Si no son necesarios, el usuario no tiene por qué interactuar con ellos, y puede limitarse a traducir el juego directamente. En cualquier otro caso, deberá seguir los siguientes pasos:

1. Primero, cada vez que se quiera escribir una palabra que variará según el género del sujeto que aún queda por definir, hay que escribir en los archivos .xml del lenguaje de texto con este formato:

Primero se escribe todo entre dos corchetes cuadrados -> []

Se pone el nombre de la variable entre comillas con dos puntos al final -> [“nom”:]

Por último se ponen las opciones, separadas por la barra | -> [“nom”:Opción1|Opción2]

Siendo **nom** un nombre que se escribirá para definir el género de un personaje, y las **Opciones** separadas por | los distintos formatos que tendrá el texto según se cambie de género al sujeto. Por ejemplo:

```
<text id="9">[“prota”:Él|Ella] está muy content[“prota”:o|a]</text>
```

Siendo **prota** el nombre de la variable, si el género dado al personaje protagonista fuera femenino (En este caso representado por un 1) la frase final sería: “Ella está muy contenta”

2. Lo único que quedaría sería definir el género de las variables, que será representado con un int. Para ello, existe un método público en "**Pacolization.cs**" llamado "**WriteGenderConf(string key, int value)**" siendo "**key**" el nombre de la variable que se deberá poner al principio entre comillas y "**value**" el género.

Queda aclarar que el valor de género representado por un int pillaré esa posición como opción entre las dadas (Las posiciones serían las siguientes: ["nom"0|1|2|3|4|5]) Por lo tanto hay que aclarar a la hora de redactar si la posición 0 es el género masculino, femenino... e igual para el resto de posiciones. Por lo general, tan solo se utilizarán las posiciones 0 y 1 para masculino y femenino, aunque el programa es escalable, y puedes añadir una posición 2 para lenguaje neutro, por ejemplo. En el caso de que el value sobresalga de las posibles posiciones (siendo su valor, por ejemplo, 100 o -1) o si no se encuentra una variable con la key dada, se tomará la posición 0 por defecto.

```
public void WriteGenderConf(string key, int value)
{
    Li.WriteGenderConfToXML(key,value,filePath[currentLang]);
}
```

Cómo utilizar el sistema de variables

El sistema de variables está también implementado dentro de la lectura de los archivos de idiomas. El usuario también puede no utilizarlos, aunque pueden ser muy útiles para incluir en los diálogos completos cosas variables como nombres de personajes, objetos variados... Para utilizarlos, se deberán seguir los siguientes pasos:

1. Primero, cada vez que se quiera implementar una palabra que variará según el valor que se haya guardado, se deberá escribir con este formato

Primero se escribirá todo entre dos corchetes con una exclamación antes -> **!{ }**

Y luego se escribirá el nombre clave de la variable que deba sustituir al texto -> **!{nom}**

Automáticamente, el texto será sustituido por el valor de tipo string que se haya asociado al nombre de variable **nom**

2. Lo único que quedaría sería asociar un valor string a el nombre de variable nom. Para ello, existe un método público en "**Pacolization.cs**" llamado "**WriteVariables(string key, string value)**" siendo "**key**" el nombre de la variable que se deberá poner entre los corchetes y "**value**" el valor que guardará la variable.

En el caso de que no se encuentre una variable con la key dada, la variable será quitada sin ser sustituida por ningún tipo de texto.

```
public void WriteVariables(string key, string value)
{
    Li.WriteVariables(key,value,filePath[currentLang]);
}
```

Cómo utilizar el sistema monetario

Dependiendo de la región en la que se juegue a los juegos, estos tienen un tipo de sistema monetario distinto. Además, los separadores decimales también cambian de signo dependiendo de la localización del juego.

Estos modificadores están implementados en los archivos de configuración del idioma correspondiente, justo después de definir la configuración para los textos de la siguiente forma..

```
<Lenguaje>
  <Nombre>es</Nombre>
  <Texto>
    <Direccion>Iz_Der</Direccion>
  </Texto>
  <Numeros>
    <Decimal>,</Decimal>
    <SeparadorMil>.</SeparadorMil>
    <Dinero>€</Dinero>
    <PosicionMoneda>3</PosicionMoneda>
  </Numeros>
</Lenguaje>
```

Decimal -> Es el signo de puntuación que se utiliza para separar las cantidades monetarias. Por ejemplo, en el Euro para separar el precio de 100 euros y 50 céntimos se hace usando una coma: 100,50 €.

SeparadorMil -> Signo de puntuación que se utiliza para separar los numeros en unidades de más de 3 cifras. Por ejemplo, para la expresar la cantidad de mil euros se utiliza un punto: 1.000 €

Dinero -> El símbolo del valor monetario que se pondrá al lado de la cantidad.

PosicionMoneda -> Posición en la que se situará el símbolo monetario. Por ejemplo, los euros se suelen poner al final de la cantidad: 100 €, mientras que las libras se ponen delante: £100.

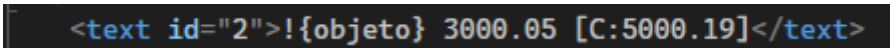
Todos estos atributos se les asigna a un objeto **NumberFormatInfo** de la librería de .NET de C# y se almacena en la clase LocalCore para ser utilizado de manera global

Si queremos mostrar números grandes con separadores de miles o decimales de forma limpia no hace falta utilizar una etiqueta especial . Simplemente basta con poner el número directamente en el archivo .xml, aunque cabe aclarar que siempre utilizar como separador el punto (.) durante el desarrollo. Nuestro sistema se encargará de transformarlo después al formato destino.

Por otro lado, si queremos implementar un número que represente una cantidad de dinero o un precio y que deba llevar su símbolo de divisa y posición correspondiente, se debe escribir de la siguiente manera:

- Se escribe una C con dos puntos entre corchetes cuadrados -> **[C:]**
- Después se escribe el número o valor de la moneda en su interior, por ejemplo si queremos poner la cantidad 5000,19 -> **[C: 5000.19]**

Nuestro sistema se encargará de escanear nuestro texto con esta nomenclatura y reemplazar las coincidencias, escribiendo **5.000,19 €** si el idioma configurado es el español por ejemplo. Aquí una imagen aplicando los dos casos explicados:



```
<text id="2">{objeto} 3000.05 [C:5000.19]</text>
```

Cómo utilizar sistema de cantidades (singulares y plurales)

El sistema de singulares y plurales está implementado dentro de la lectura de archivos de los idiomas. El usuario puede elegir entre utilizarlo o no, pero es muy útil para incluir en diálogos textos que cambian según el número de objetos, monedas o elementos que tenga el jugador.

Cada vez que se quiera implementar una palabra o frase que varíe según la cantidad numérica, se debe escribir en los archivos .xml de idioma correspondiente de la siguiente manera:

- Primero se escribe una P con dos puntos dentro de dos corchetes cuadrados -> **[P:]**
- Después se pone el nombre de la variable entre comillas con otros dos puntos al final -> **[P: "nom"]**
- Por último se ponen las opciones del texto a mostrar dependiendo de la cantidad separadas por un separador | -> **[P: "nom":Opcion1|Opcion2]**

Siendo **nom** el nombre de la variable que registra la cantidad y las **OpcionX** los distintos textos que se mostrarán según el número guardado, el sistema procesa el texto de distintas formas según las opciones que le des:

- Si pones 2 opciones, la opción 1 se usará si la cantidad es exactamente igual a 1 (singular) y la opción 2 se usará en cualquier otro número(plural, incluyendo el 0).
- Si pones 3 opciones, la opción 1 se usa cuando la cantidad es 0, la opción 2 si es exactamente igual a 1 y la opción 3 si es 2 o más.

Por ejemplo, si pongo esto en mi archivo .xml:

```
<text id="4">Tienes !{oro} [P:"oro":moneda|monedas] de oro
```

Siendo oro el nombre de la variable, si la cantidad de oro fuese 1, la frase final sería **“Tienes una moneda de oro”**. Si fuera 2 o más la frase cambiaría a: **“Tienes 2 monedas de oro”**.

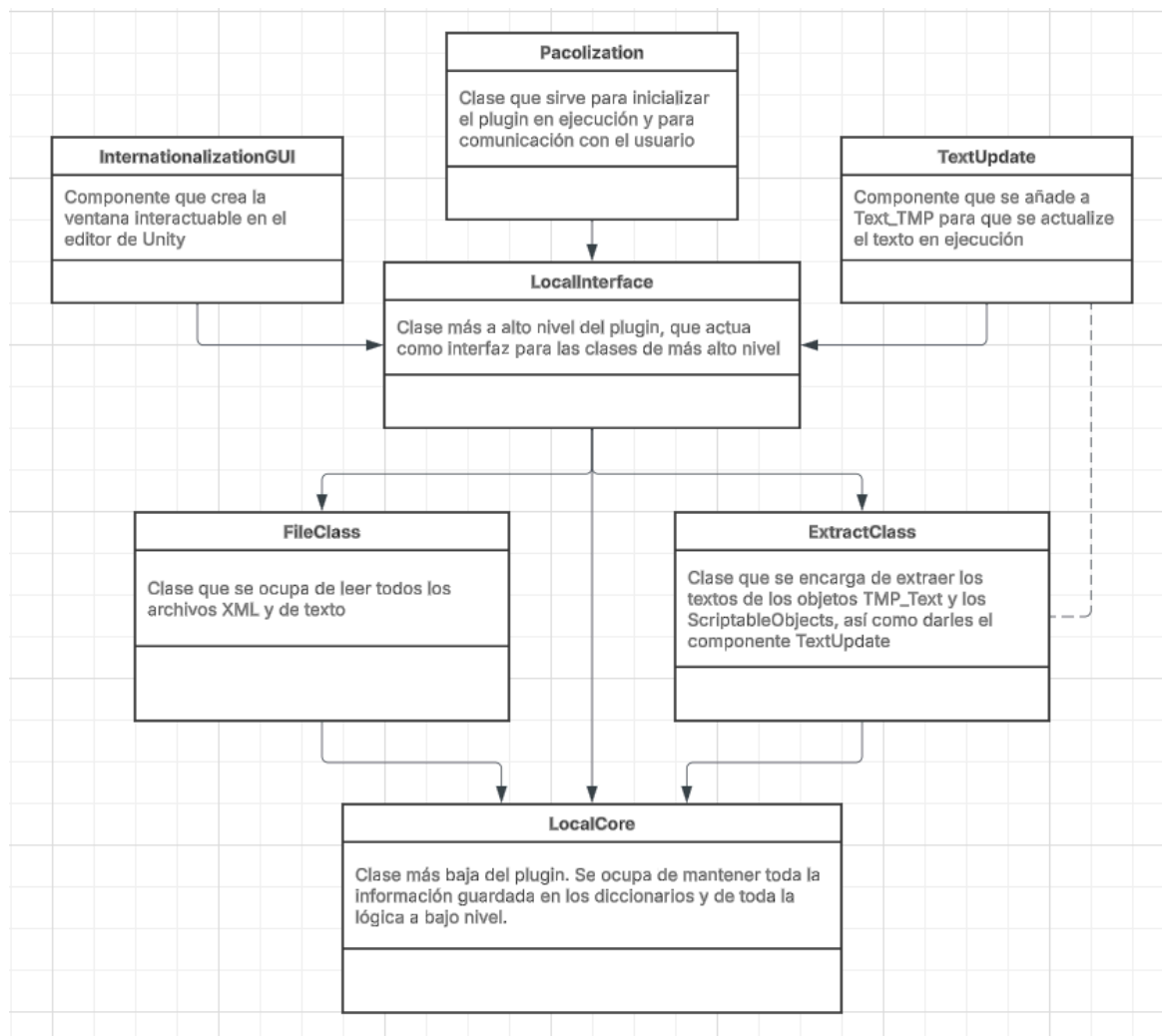
Para definir la cantidad de variables cuantificadoras desde nuestro código existe un método público en **LocalInterface** llamado **WriteCantidades**

```
public void WriteCantidades(string key, int value, string path)
```

- **key:** Es el nombre de la variable entre comillas definida en el archivo xml
- **value:** El número entero con la cantidad actual de la variable (el sistema lo guarda en un diccionario interno llamado variables para consultarlo).
- **path:** La ruta del archivo xml para que el sistema vuelva a leerlo y actualice el texto con el plural o singular correcto de forma automática.

3.3 Implementación

Diagrama de Clases



El “UI Clamper” (clase encargada de mantener los textos dentro de un margen, ya sea dentro del cuadro de texto o de la misma pantalla) no tiene porqué interactuar con el resto de componentes del diagrama de clases para que la extracción y procesamiento de textos funcione, pero será llamado desde el **ExtractClass**. Concretamente desde el método “**GUISet**” que lo que hace es añadir el “UI Clamper” a los textos, pero por lo demás no hay más comunicación.

Archivos XML

El plugin accede a dos archivos xml por cada idioma que esté implementado. Un archivo de configuración, que contiene información que el usuario puede variar para cada idioma; separadores decimales y de mil unidades, símbolos de dinero y dirección en la que se escribe el texto. Y luego está el archivo principal, que recoge todos los textos de las escenas de Unity y le pone un ID a cada uno para identificarlo. Se pueden ver los IDs en los **TextUpdate** implementados en cada Texto de la escena. Estos archivos xml son los que se podrán modificar para así variar la interfaz de una escena según su idioma.

Ejemplo de un archivo de este estilo:

```
<?xml version="1.0" encoding="UTF-8"?>
<translations lang="es">
  <text id="0">Inicio</text>
  <text id="1">Menu</text>
  <text id="2">Boton</text>
  <text id="3">Boton</text>
  <text id="4">Hola</text>
  <text id="5">Buenos dias</text>
  <text id="6">Boton</text>
  <text id="7">6</text>
  <text id="8">7</text>
  <text id="9">asdf</text>
</translations>
```

4. Resultados Obtenidos

4.1 Realización de pruebas

Aquí un vídeo de las pruebas realizadas en el videojuego “Civilízame Esta”:

<https://youtu.be/0xWpTfUntro>

Para estas pruebas se ha mostrado la importación del plugin a un proyecto grande de Unity, el objeto que se debe crear con la clase “Pacolization”, los apartados que se deben rellenar en esta clase, el archivo xml que se genera con todos los diálogos y textos, cómo su modificación alteraría los objetos del proyecto, y un ejemplo del uso de los modificadores de género, y cómo alterar durante la ejecución dicho valor puede modificar el texto que se muestra

Aquí un video de las pruebas realizadas en el videojuego “El custodio de Babel”:

<https://youtu.be/z98sOTqagjQ>

Para las siguientes pruebas se ha mostrado la versatilidad de nuestro plugin y cómo se maneja el cambio en tiempo real de idiomas. Se ha comprobado que el cambio de idiomas persiste entre escenas y hasta se ha comprobado que el cambio de género también.

4.2 Cambios en la extraordinaria

- Se ha limpiado la memoria para hacerla más accesible al usuario, centrándonos más en qué podemos hacer y en cómo y menos en los detalles técnicos del código.
- Hemos investigado cómo funcionan los sistemas de internacionalización en Unity y Unreal 5
- Hemos implementado múltiples modificadores del texto en ejecución (Modificadores de género, sistema de variables, sistema monetario y sistema de cantidades) de manera que sean prácticos, no intrusivos y sencillos de utilizar
- Hemos organizado el proyecto mejor, reforzando la estructura de las clases, separando cada idioma y fichero de configuración en un archivo .xml separado para facilitar su modificación y evitando cualquier acción innecesaria, como la persistencia de información que se podría haber almacenado dentro del propio código.
- Se han limpiado los archivos de configuración de idiomas eliminando todo lo que no se utilizaba o era innecesario.

5. Conclusiones

Después de un gran trabajo de investigación y varias pruebas creemos que nuestro plugin final consigue adaptarse a estándares profesionales y logra compararse con otras herramientas que son usadas en la industria.

Con respecto a lo que debíamos cambiar, ya se han implementado el uso de modificadores de género, de variables y derivados, lo cuál faltó en la entrega ordinaria. También se han separado todos los textos de cada lenguaje en un archivo .xml separado del resto, y se han investigado encarecidamente herramientas similares, viendo así por qué realizan las cosas de cierta manera, tomando inspiración de ello.

Tras ver la diferencia entre esta entrega con su primera versión, hemos observado cómo el proyecto ha madurado a algo más profesional que una simple práctica. Estamos convencidos de que podríamos utilizar de forma competitiva este plugin para nuestros proyectos tan bien como con cualquier otra herramienta similar más conocida.

6. Adenda

6.1 Aportaciones en la ordinaria

- **Jiale He:** Durante el proyecto he aportado trabajo en el diseño de arquitectura en cuanto a cómo extraer strings de los proyectos y cómo manejar el cambio de texto en runtime a través del TextUpdate. También he estado gran parte del proyecto debuggeando y comprobando el correcto funcionamiento de todos los aspectos del proyecto. En general me he encargado de la clase ExtractClass haciendo un gran trabajo de investigación en cuanto a la extracción de textos de ScriptableObjects y la obtención de TMP_Texts.

En el proyecto me he encargado también del correcto funcionamiento del plugin en runtime con el cambio de idiomas y la recolección de referencias de ScriptableObjects. También he sido co autora del componente Pacolization.

Ayudé a debuggear a compañeros quienes no tenían acceso al compilador o a UnityEngine y acabé tocando casi todo el código, asistiendo en la solución de problemas.

- **Javier Alonso Ruiz:** En este proyecto he ayudado a la creación de la arquitectura del LocalCore. He trabajado en el archivo de configuración de idiomas "Lenguajes.xml", creándolo y ayudando a su lectura por parte del código. He ayudado a la lectura de ScriptableObjects por parte del código, al testing general del plugin. He ayudado a resolver problemas a los compañeros en sus secciones. También he aportado considerablemente a la escritura de tanto el Readme y la memoria, así como la limpieza del Readme y el paso de todos los datos de forma estructurada de este al documento de la memoria y su organización.
- **José Antonio Carmona Alfonsel:** Durante el proyecto he aportado la implementación de la interfaz del plugin en el editor (el resto del equipo la han expandido más), además de investigar cómo hacer un archivo .package para exportar el plugin y poder usarse en otros proyectos.

Me encargué de investigar e implementar herramientas que se encargan de que los textos de la UI no se salgan de los cuadros ni de la pantalla en cualquier idioma (el CLAMPING). Implementé la clase UIClamper, además del sistema que configura a través del ExtractClass todos los textos de la UI para añadirles el UIClamper y el resto de componentes de Unity que se encargan de ejercer la funcionalidad antes mencionada. Aporté documentación de cómo debería de configurarse de manera manual o

automática, además de aportar a la implementación de que dependiendo del idioma, el formato de lectura puede ser diferente.

También aporté al diseño de cómo deberíamos de extraer los textos, decidiendo que lo haríamos entre escenas de la build y scriptable objects únicamente pensando en evitar problemas de repetición de textos si los leyéramos en los prefabs. Durante el desarrollo del plugin usé el juego “El Custodio de Babel” para probar las herramientas que estaba implementando, y luego el “Stay After Class” para probarlo en un entorno en 3D y comprobar fallos.

- **Julián Serrano Chacón** Durante este proyecto he participado en el diseño de la arquitectura general del código y del funcionamiento del proyecto, contribuyendo especialmente al flujo de trabajo del plugin, incluyendo el orden de lectura de los ficheros y la forma en la que estos se leen y almacenan los datos. También he colaborado en el diseño de las estructuras de datos utilizadas para gestionar la información.
Además de haber aportado a las diferentes refactorizaciones en el diseño de la arquitectura y el código general (hemos hablado mucho de todo esto tocando todos los apa.
Además, he trabajado en la implementación de la lectura y escritura de ficheros XML correspondientes a idiomas, variables y textos, encargándome tanto del diseño de las estructuras de datos empleadas para el manejo de la información como de su implementación y organización interna.
Asimismo, he implementado distintos métodos dentro de la interfaz LocalCoreInterface para la comunicación con el core y la FileClass. En LocalCore refactoricé la estructura de almacenamiento de datos, sustituyendo un array de strings por un mapa por idioma, permitiendo así que el usuario pueda cargar en memoria únicamente el idioma que necesite si así lo desea.
También he colaborado en la implementación de la dirección del texto en TextUpdate y su registro dentro de la lista de LocalCore. Por último, he prestado apoyo al resto del equipo ayudando a resolver distintos problemas técnicos mediante llamadas de Discord.
- **Luis Javier Navarrete Pulupa:** Durante este proyecto he aportado trabajo en el diseño de la arquitectura para guardar la tabla de strings por idioma en vez de en un array de strings asociado a una ID.
También en la escritura y lectura de la tabla de strings a un archivo XML (tanto de variables, idiomas y textos extraídos), sobre todo en el diseño e implementación del archivo de textos extraídos de un juego y variables.
Por otra parte, también he colaborado en la implementación del sistema de modificadores para textos con variables definidas, con cambios grandes en FileClass (tanto en diseño como implementación).
También en el diseño e implementación de la interfaz de usuario del plugin añadiendo botones y checkBoxs y funcionalidades para extraer textos leídos desde la ventana de exploración de ficheros.

Para la aplicación de variables a un texto, he participado en la implementación de la forma para traducir los textos definidos con variables y su sustitución de valores leídos de un archivo XML.

He colaborado en la reestructuración de la estructura de datos para guardado de datos de las variables en FileClass mediante la implementación de código. También he añadido e implementado métodos en el Local Interface para poder comunicar la clase FileClass con Local Core.

He apoyado como Debugger para los que tenían dificultades con Unity.

- **Pablo Marcos Serrano:** Durante este proyecto fui pionera, poniéndome a trabajar antes que nadie para proporcionar ideas sobre una arquitectura base. Esta arquitectura fue evolucionando pero sigue teniendo trazas de la original. Hice el código original de LocalCore y participé en sus modificaciones. Además, tuve presencia en la mayoría de otras clases excepto FileClass. Me ocupé de crear una interfaz de nuestro código para el usuario y fui co-autora del componente Pacolization.

6.2 Aportaciones en la extraordinaria

- **Jiale He:** Durante la extraordinaria me he enfocado en el trabajo de investigación de otros sistemas de internacionalización de otros motores y he asistido en la documentación de esos otros sistemas. También he estado aportando principalmente a la memoria y documentación. Por último he hecho pruebas para mostrar el correcto funcionamiento de nuestro plugin.
- **Javier Alonso Ruiz:** Durante la extraordinaria he realizado trabajo de investigación sobre otros sistemas de localización, estructurando conceptualmente en base de ellos partes de nuestro plugin, y ayudando a compañeros a encontrar soluciones a problemas basándose en modelos ya creados. He documentado excesivamente la nueva memoria junto a Pablo. He desmontado el sistema de modificadores de género, lo he rediseñado y montado por completo para que sea funcional y efectivo. He ayudado en el rediseño del sistema de variables junto a Julián y Luis. He modificado Pacolización para que sea la clase a través de la cuál el usuario puede interactuar con el plugin. He grabado las pruebas del plugin con el juego "Civilízame Esta".
- **José Antonio Carmona Alfonsel:** Durante la extraordinaria me he dedicado junto a algunos de mis compañeros a hacer que los textos en diferentes idiomas, en vez de que se guarden y se lean en un mismo fichero, como estaba hecho en la ordinaria, ahora están repartidos en diferentes ficheros, de tal manera que los textos en español están en un .xml diferente a los textos en inglés por ejemplo.

También me he encargado de que se separen las configuraciones de cada idioma en diferentes ficheros, y que no estén en uno solo, y hemos tenido que modificar cosas del LocalCore para que el mapa de textos, en vez de

guardar diferentes idiomas, ahora se guarden los textos de un único idioma. También he refinado y resumido la documentación del UI Clamping para que se sepa mejor para qué sirve y cómo se configura.

- **Julián Serrano Chacón** Durante la extraordinario me he dedicado principalmente al código junto a Luis Javier, Javier Alonso y Jose Antonio. Mis tareas han sido las de separar en diferentes ficheros los textos y las configuraciones de cada idioma, refactorizar el manejo de variables para que ya no usen ficheros y funcionen puramente por código, permitir la lectura de plurales y he ayudado a Javier a hacer que todo se integre correctamente y cada texto se modifique correctamente, además hemos movido la información de FileCLass a LocalCore para tener mayor coherencia de que hace cada clase.
- **Luis Javier Navarrete Pulupa:** Durante la convocatoria extraordinaria he estado trabajando en el rediseño del nuevo sistema de variables dinámicas junto a Julián Serrano y Javier Alonso, además de la implementación del sistema de formato de monedas, permitir el uso de palabras en singular y plural y el género asociado a una variable determinada. También he colaborado en la separación de un fichero de configuración de un idioma en un archivo distinto y en el uso de las variables de dichos archivos(como el Decimal o Dinero) para que ejecuten la lógica de escritura de una forma distinta dependiendo del idioma.
- **Pablo Marcos Serrano:** Debido a que había hecho más trabajo en la ordinaria mi trabajo aquí ha sido más breve. Me he dedicado principalmente a revisar el trabajo de los demás, señalando fallas y proponiendo soluciones, y confirmándolo como completado. Aparte de eso he contribuido en casi todas las partes de la nueva memoria.